

Занятие 2. Как редактировать конечную память: decay, delta rule, gates

1. Конечная матричная память

Пусть состояние модели — матрица

$$S_t \in \mathbb{R}^{d_v \times d_k},$$

где d_k — размерность ключа, а d_v — размерность значения.

Ключ k_t и запрос q_t лежат в $\mathbb{R}^{d_k}$, значение v_t — в $\mathbb{R}^{d_v}$.

Additive write

Простейшая запись в состояние — добавить внешнее произведение значения и ключа:

$$S_t = S_{t-1} + v_t k_t^\top.$$

Внешнее произведение $v_t k_t^\top$ имеет ту же форму $d_v \times d_k$, что и память.

Чтение выполняется умножением памяти на запрос:

$$y_t = S_t q_t.$$

Если запрос совпадает с ключом записи, память извлекает компоненту состояния, лежащую в направлении этого ключа.

Главное свойство такой памяти — **фиксированный shape**. Число элементов в S_t не зависит от длины последовательности. После десяти и после миллиона токенов хранится матрица одной и той же формы.

Это принципиально отличается от KV-cache обычного attention: KV-cache хранит ключи и значения прошлых позиций отдельно, поэтому растёт вместе с длиной контекста. Конечное состояние вместо этого сжимает всю историю в фиксированное число скаляров.

Фиксированный размер состояния решает проблему роста памяти при авторегрессивной генерации, но не означает неограниченную информационную ёмкость.

2. Интерференция записей

Если в additive state записано несколько ассоциаций,

$$S = \sum_{i=1}^n v_i k_i^\top,$$

то чтение по некоторому ключу k_j даёт

$$S k_j = \sum_{i=1}^n (k_i^\top k_j) v_i.$$

Отделяя собственную запись от остальных, получаем

$$S k_j = \|k_j\|_2^2 v_j + \sum_{i \neq j} (k_i^\top k_j) v_i.$$

Для единичного k_j первый член равен нужному значению v_j , а вторая сумма — **interference**, то есть примесь чужих записей.

Геометрическая причина интерференции

Коэффициент $k_i^\top k_j$ измеряет перекрытие двух ключей.

- Если ключи ортогональны, $k_i^\top k_j = 0$, поэтому запись i не влияет на чтение по k_j .
- Если ключи похожи, их скалярное произведение велико, и чужая запись заметно попадает в результат.
- Если ключи совпадают и нормированы, чужая запись попадает полностью.

Интерференция — не то же самое, что забывание. При забывании нужная информация исчезла. При интерференции нужная информация всё ещё присутствует, но к ней примешиваются другие компоненты.

Пример: если состояния для «Маша» и «Миша» получают похожие ключи, записи «Маша любит кофе» и «Миша любит чай» могут смешиваться при чтении. Память выполнила additive update правильно; проблема возникает из геометрии самих ключей.

Почему большая размерность не устраняет проблему

В высокой размерности случайные векторы действительно обычно почти ортогональны. Но одновременно растёт число записей. Каждый отдельный чужой вклад может быть мал, однако их сумма уже не обязана быть малой.

Для независимых случайных единичных ключей и гауссовых значений ошибка чтения первой записи имеет вид

$$e = \sum_{i=2}^n (k_i^\top k_1) v_i.$$

При этих предположениях

$$\mathbb{E} \|e\|_2^2 = (n-1) \frac{d_v}{d_k}.$$

Следовательно, характерный RMS-масштаб интерференции равен

$$\text{RMS}(e) = \sqrt{(n-1) \frac{d_v}{d_k}}.$$

Из формулы следуют важные ориентиры:

- число записей увеличивает типичный шум примерно как $\sqrt{n}$;
- увеличение d_k уменьшает его как $1/\sqrt{d_k}$;
- увеличение d_v увеличивает норму шума как $\sqrt{d_v}$;
- при $d_k = d_v$ остаётся масштаб порядка $\sqrt{n-1}$.

Это модель для случайных ключей, а не прогноз качества обученной нейросети. Обученная модель сама формирует ключи, поэтому их распределение может быть совершенно другим.

Для случайных экспериментов важно измерять типичную ошибку по множеству реализаций, например RMS. Один случайный запуск способен заметно отклоняться от среднего и сам по себе ничего не устанавливает.

3. Ёмкость состояния фиксированного размера

Требование точно хранить пары (k_i, v_i) можно записать как

$$S k_i = v_i.$$

Рассмотрим одну строку S . В ней d_k неизвестных, а каждая новая произвольная ассоциация добавляет ещё одно линейное уравнение.

Если имеется больше d_k произвольных независимых требований, система в общем случае становится переопределённой. Поэтому одна матрица с ключами размерности d_k не способна точно реализовать неограниченное число произвольных независимых ассоциаций.

Если ключей не больше d_k и они линейно независимы, точное состояние может существовать даже при неортогональных ключах. Но простая additive-запись не обязана это состояние находить.

Особый простой случай — ортонормированные ключи. Тогда

$$k_i^\top k_j = \delta_{ij},$$

и additive state воспроизводит записанные значения без взаимных примесей.

Отсюда важно различать:

- **существование** матрицы S , удовлетворяющей всем ассоциациям;
- **правило обновления**, которым модель пытается получить эту матрицу;
- **фактическую ёмкость всей нейросети**, где есть много голов, слоёв и структурированные данные.

Граница порядка d_k относится к произвольным независимым ассоциациям в одной такой матрице, а не является буквальным числом «фактов», которые способна помнить вся модель.

4. Что означает длинный контекст

Фраза «контекст модели — миллион токенов» может скрывать разные свойства.

Модель принимает длинный вход

Это означает лишь, что такой вход проходит через интерфейс, буферы и вычислительный граф.

Модель сохраняет работоспособность

Она продолжает выдавать связный результат на большой длине, а вычисления не становятся численно или алгоритмически бессмысленными.

Модель использует далёкое начало

Она способна извлекать и применять информацию, находящуюся далеко в прошлом.

Каждое из этих утверждений сильнее предыдущего и из него автоматически не следует.

Эффективная длина контекста

Эффективная длина контекста — диапазон длин, на котором качество на интересующей задаче остаётся приемлемым.

Она не равна:

- заявленному размеру окна;
- размеру входного буфера;
- физической длине KV-cache;
- масштабу экспоненциального decay;
- максимальной позиции, на которой один раз удалось извлечь факт.

5. Инструменты измерения длинного контекста

Needle in a haystack

В длинный текст вставляется один известный факт, после чего проверяется возможность извлечь его с определённой позиции.

Такой тест отвечает прежде всего на вопрос: **может ли модель достать один удалённый факт?**

Он не измеряет напрямую ёмкость памяти при одновременном хранении множества ассоциаций.

MQAR — Multi-Query Associative Recall

MQAR помещает в последовательность множество пар и множество событий recall на различных расстояниях.

Он нагружает именно способность поддерживать множество ассоциаций в одном контексте и поэтому лучше выявляет ограничения конечного состояния.

MQAR не выдаёт универсальное число «ёмкость памяти = N». Результат зависит от:

- числа пар;
- расстояний между записью и запросом;
- словаря;
- числа запросов;
- устройства модели;
- обучения;
- размера состояния.

Прикладные наборы задач

Для реальной системы конечной целью остаётся качество на собственном распределении: документах, коде, диалогах, договорах и других целевых данных.

Needle, MQAR и прикладной benchmark измеряют разные свойства, поэтому результат одного теста нельзя автоматически переносить на другой.

Как проектировать эксперимент

Для дальнего извлечения имеет смысл менять позицию факта или расстояние до запроса, сохраняя остальные условия.

Для исследования ёмкости нужно отдельно изменять:

- число ассоциаций;
- длину последовательности;
- расстояние до recall;
- размер конечного состояния.

Остальные факторы желательно фиксировать.

Обычный attention полезен как контрольная архитектура без того же узкого места фиксированного state. Это не «идеальная память», а контроль с другим способом хранения истории.

6. Fixed decay: глобальное забывание

Первый способ управлять конечной памятью — перед новой записью ослаблять всё старое состояние:

$$S_t = \gamma S_{t-1} + v_t k_t^\top, \quad 0 \leq \gamma \leq 1.$$

Параметр γ определяет retention старой памяти.

- $\gamma = 1$ — ничего не забывается.
- γ близко к 1 — память затухает медленно.
- γ близко к 0 — почти всё старое быстро стирается.

Разворачивая recurrence, получаем

$$S_t = \sum_{i=1}^t \gamma^{t-i} v_i k_i^\top.$$

Свежая запись имеет вес 1 , предыдущая — γ , ещё более старая — γ^2 и т. д.

Это экспоненциальное забывание: жёсткой границы окна нет.

Суммарная масса памяти

Для $0 \leq \gamma < 1$ сумма коэффициентов ограничена геометрической прогрессией:

$$\sum_{r=0}^{t-1} \gamma^r = \frac{1 - \gamma^t}{1 - \gamma} \rightarrow \frac{1}{1 - \gamma}.$$

Если отдельные записи ограничены по величине, fixed decay не позволяет их общей массе бесконтрольно расти.

Но ограниченная норма состояния ещё не означает хорошую память: состояние может быть небольшим по величине и при этом потерять необходимую информацию.

Масштаб памяти

Удобная характеристика decay:

$$\tau = \frac{1}{1 - \gamma}.$$

При γ близком к 1 на расстоянии порядка τ вклад старой записи уменьшается примерно до e^{-1} исходного значения.

τ — не эффективная длина контекста. Это только характеристика скорости экспоненциального затухания.

Half-life

Период полураспада h определяется условием

$$\gamma^h = \frac{1}{2}.$$

Отсюда

$$h = \frac{\log(1/2)}{\log \gamma}.$$

Для γ близкого к 1 выполняется приближение

$$h \approx \frac{\ln 2}{1 - \gamma} \approx 0.693\tau.$$

Half-life также не создаёт жёсткого окна: после h шагов запись не исчезает, а только теряет половину веса.

7. Input-dependent decay

Гейт забывания необязательно должен быть постоянным. Можно вычислять его отдельно для каждого токена:

$$S_t = \gamma_t S_{t-1} + v_t k_t^\top.$$

Тогда запись i в состоянии S_t имеет коэффициент

$$w_{i \rightarrow t} = \prod_{j=i+1}^t \gamma_j.$$

Полное состояние можно записать как

$$S_t = \sum_{i=1}^t \left(\prod_{j=i+1}^t \gamma_j \right) v_i k_i^\top.$$

Это позволяет модели самой выбирать, где хранить прошлое долго, а где резко его сбрасывать.

Например, на границе двух независимых документов значение γ_t около нуля почти уничтожает всё состояние, накопленное до этой границы.

Input-dependent decay — механизм, а не самостоятельная архитектура. Наличие зависящего от входа гейта само по себе не превращает модель в Mamba: в Mamba от входа зависит более широкая система параметров состояния и используется свой способ вычисления операции.

Ограничение decay

Decay отвечает на вопрос **когда забывать**, но почти не отвечает на вопрос **что именно исправить**.

Глобальный множитель ослабляет всё состояние одновременно. Если один факт устарел, а остальные всё ещё нужны, уменьшение γ повредит и нужным записям.

Пример: после утверждения «Алиса живёт в Риме» появляется «Алиса переехала в Париж». Fixed decay ослабит старый Рим только потому, что он старше, но одновременно ослабит и остальные старые факты.

8. Один decayed operator можно вычислять разными алгоритмами

RetNet удобен как пример того, что один математический оператор допускает несколько точных способов вычисления.

Рекуррентная форма

$$S_t = \gamma S_{t-1} + v_t k_t^\top.$$

$$y_t = S_t q_t.$$

Для генерации она удобна тем, что между токенами нужно хранить только конечный state.

Параллельная форма

Определяется causal retention mask

$$R_{ti}^{(\gamma)} = \mathbf{1}[i \leq t] \gamma^{t-i}.$$

Тогда тот же выход можно представить непосредственно как

$$y_t = \sum_{i=1}^t \gamma^{t-i} (k_i^\top q_t) v_i.$$

Все позиции можно обрабатывать параллельно, но внутри появляется таблица размера «длина × длина». Поэтому **параллельно** не означает **дёшево**.

Chunkwise-форма

Последовательность разбивается на блоки.

- Внутри блока используется параллельный расчёт.
- Между блоками переносится только конечное состояние.
- Состояние прошлого блока ослабляется на количество шагов, прошедших до нужной позиции нового блока.
- Новые записи внутри блока получают decay в соответствии со своим возрастом.

Это точная перестановка вычислений того же оператора, а не приближение.

Chunkwise-подход особенно важен на GPU: большие матричные операции внутри блока хорошо параллелятся, а размер межблочной памяти остаётся фиксированным.

Sanity check для decay

При $\gamma = 1$ должны восстановиться формулы обычного causal linear attention без затухания.

Это полезный тест корректности реализации.

Переменные гейты

При γ_t , зависящих от позиции, степени заменяются произведениями.

Для положительных гейтов произведения удобно вычислять через логарифмы:

$$\prod_j \gamma_j = \exp \left(\sum_j \log \gamma_j \right).$$

Это полезно одновременно для параллелизации и численной устойчивости.

9. Delta rule: read $\rightarrow$ error $\rightarrow$ correction

Decay не умеет выборочно заменить конкретную ассоциацию. Для этого используется **delta rule**.

Перед записью память сначала читается по текущему ключу:

$$\hat{v}_t = S_{t-1} k_t.$$

Затем вычисляется ошибка предсказания:

$$e_t = v_t - \hat{v}_t.$$

После этого записывается не всё v_t , а только correction:

$$S_t = S_{t-1} + \beta_t (v_t - S_{t-1} k_t) k_t^\top.$$

β_t задаёт силу исправления.

- Если память уже возвращает v_t , ошибка равна нулю и состояние не изменяется.
- Если ответ неверен, изменение направлено только на недостающую разницу.

- При повторном сообщении уже правильно записанного факта состояние не обязано продолжать бесконтрольно расти.

В реализации порядок принципиален:

1. прочитать **старое** состояние;
2. вычислить ошибку;
3. обновить состояние;
4. использовать уже обновлённое состояние для последующего чтения текущего шага, если оператор определён именно так.

10. Delta rule как online gradient descent

Состояние можно рассматривать как веса маленькой линейной модели:

$$k_t \mapsto Sk_t.$$

Для текущей пары (k_t, v_t) определим локальный squared loss:

$$\ell_t(S) = \frac{1}{2} \|Sk_t - v_t\|_2^2.$$

Его градиент по матрице состояния:

$$\nabla_S \ell_t = (Sk_t - v_t)k_t^\top.$$

Один шаг градиентного спуска с learning rate β_t :

$$S_t = S_{t-1} - \beta_t (S_{t-1}k_t - v_t)k_t^\top.$$

После изменения знака внутри скобок получаем delta rule:

$$S_t = S_{t-1} + \beta_t (v_t - S_{t-1}k_t)k_t^\top.$$

Таким образом, внутри слоя присутствует маленький online learner.

Важно различать два уровня обучения:

- **network weights** вычисляют ключи, значения, гейты и другие параметры и обучаются внешним оптимизатором;
- **fast state** S_t меняется непосредственно во время обработки последовательности.

β_t играет роль шага обучения fast state.

Исторически это связано с правилом Widrow–Hoff/LMS: линейная модель исправляется по ошибке текущего примера.

11. Delta rule как erase + write

Раскроем update:

$$S_t = S_{t-1} + \beta_t v_t k_t^\top - \beta_t S_{t-1} k_t k_t^\top.$$

Или

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Здесь видны две операции.

Write

$$+ \beta_t v_t k_t^\top.$$

Это запись нового значения.

Key-directed erase

$$- \beta_t S_{t-1} k_t k_t^\top.$$

Она удаляет часть старого состояния, связанную с текущим направлением ключа.

Для единичного k_t матрица $k_t k_t^\top$ является проектором на направление k_t .

При $\beta_t = 1$

$$I - k_t k_t^\top$$

полностью удаляет компоненту старого состояния вдоль этого направления, оставляя ортогональные направления неизменными.

Именно erase отличает delta rule от additive write с простым коэффициентом.

12. Когда delta correction становится точной

Пусть до записи память возвращала

$$\hat{v}_t = S_{t-1}k_t.$$

После delta update чтение по тому же ключу:

$$S_t k_t = \hat{v}_t + \beta_t \|k_t\|_2^2 (v_t - \hat{v}_t).$$

Это интерполяция между старым предсказанием и target с эффективной силой

$$\beta_t \|k_t\|_2^2.$$

Точная правка за один шаг получается при

$$\beta_t = \frac{1}{\|k_t\|_2^2}.$$

Для единичного ключа условие упрощается:

$$\|k_t\|_2 = 1, \quad \beta_t = 1.$$

Тогда

$$S_t k_t = v_t.$$

Поэтому нормировка ключей делает β_t непосредственно интерпретируемой как силу замены.

Если $\beta_t = 0$, запись отсутствует.

Для единичного ключа и $0 < \beta_t < 1$ память проходит только часть пути от старого ответа к новому.

Если шаг слишком велик, возникает overshoot. Ошибка по текущему ключу после update масштабируется как

$$e_t^+ = (1 - \beta_t \|k_t\|_2^2) e_t.$$

Поэтому слишком большое произведение $\beta_t \|k_t\|_2^2$ способно вызвать колебания или расходимость. Нормированные ключи и ограниченная β_t дают простую устойчивую интерпретацию.

В опубликованной форме Gated DeltaNet ключи и запросы нормируются, но нормировка не является математически обязательной для любой delta-like архитектуры.

13. Targeted correction не означает isolated correction

Delta rule адресно исправляет направление текущего ключа, но не изолирует его от всех остальных направлений.

Для другого запроса u :

$$S_t u = S_{t-1} u + \beta_t (v_t - \hat{v}_t) (k_t^\top u).$$

Размер побочного изменения определяется перекрытием $k_t^\top u$.

- Если $u \perp k_t$, чтение по u не меняется.
- Если u похож на k_t , изменение заметно.
- Чем сильнее похожи ключи, тем сильнее исправление одного факта затрагивает другой.

Это фундаментальный компромисс: delta rule более локальна, чем глобальный decay, но геометрическая интерференция не исчезает.

Если два близких ключа обозначают один и тот же изменившийся факт, сильная взаимная коррекция полезна: старое значение действительно нужно заменить.

Если близкие ключи обозначают два разных факта, исправление одного может повредить второй.

14. Delta rule не равна нормализации linear attention

У нормализованного additive linear attention запись в состояние остаётся обычной:

$$S = \sum_i v_i k_i^\top.$$

Дополнительно накапливается масса ключей:

$$z = \sum_i k_i.$$

Чтение выполняется как

$$y(q) = \frac{Sq}{z^\top q}.$$

Раскрывая суммы:

$$y(q) = \frac{\sum_i (q^\top k_i) v_i}{\sum_i (q^\top k_i)}.$$

Если все веса $q^\top k_i$ положительны, это нормированная взвешенная сумма. Простым средним она становится только при равных весах.

При отрицательных сходствах интерпретация обычного выпуклого среднего вообще перестаёт работать.

Главное различие

Нормализация меняет чтение, но не состояние.

В S по-прежнему лежит additive sum всех записей.

Delta rule меняет само состояние.

Она вычисляет ошибку

$$v_t - S_{t-1} k_t$$

и на её основе редактирует память.

Зависимость от порядка

Additive accumulation и сумма ключей коммутативны, поэтому нормализованное чтение не зависит от порядка записей.

Delta rule зависит от порядка: каждое новое correction вычисляется из состояния, созданного предыдущими corrections.

Когда delta возвращает «последнее значение»

Это утверждение верно только при специальных условиях:

- читаем тем же ключом, которым только что записывали;
- выполняется $\beta \|k\|_2^2 = 1$.

В общем случае delta rule не является механизмом «всегда вернуть последнее значение».

15. Delta rule и метод Качмажа

Побочное изменение ранее записанной ассоциации не означает, что точное совместное решение невозможно.

Если требуется одновременно выполнить

$$Sk_1 = v_1, \quad Sk_2 = v_2, \quad \dots,$$

то один delta update решает только локальную задачу для текущего ключа.

При единичных ключах correction можно интерпретировать как проекцию текущего решения на множество состояний, удовлетворяющих очередному линейному ограничению.

Если циклически возвращаться к ограничениям и выполнять такие corrections снова, получается связь с **методом Качмажа** для решения систем линейных уравнений.

Для согласованной системы повторные проходы могут приближать состояние к совместному решению.

Это подчёркивает важное различие:

- точное состояние может существовать;
- один online-проход delta rule не обязан его найти;
- в обычном тексте нет гарантии, что нужные ключи будут многократно повторяться в удобном порядке.

16. Почему delta rule сложнее параллелизовать, чем decay

Для fixed decay каждый шаг умножает старое состояние на скаляр γ . Скаляры коммутируют, поэтому всю историю decay легко свести к степеням или произведениям.

В delta rule transition старого состояния содержит матрицу

$$E_t = I - \beta_t k_t k_t^\top.$$

Последовательность обновлений содержит произведение

$$E_1 E_2 \cdots E_t.$$

В общем случае

$$E_i E_j \neq E_j E_i.$$

Порядок corrections важен, поэтому простая causal mask со степенями одного числа уже не даёт дешёвую полностью параллельную формулу.

Рекуррентная форма

Delta rule естественно вычисляется token за tokenом и требует только текущего state.

Chunkwise-форма

Точный расчёт по блокам тоже возможен, но требует более сложной алгебры.

WY representation

Последовательность простых rank-one transformations можно компактно представить через узкие матрицы вместо явного хранения и перемножения больших transition matrices.

WY-представление используется для организации точного chunkwise-вычисления DeltaNet.

Это позволяет внутри блока снова свести значительную часть работы к большим матричным операциям, подходящим для GPU.

Практический вывод: простота state equation ничего не говорит о простоте эффективного обучения. Для архитектуры важен не только transition, но и алгоритм его вычисления.

17. Зачем нужны две независимые ручки

Decay и delta rule решают разные задачи.

Глобальное forgetting

Нужно, когда большой фрагмент прошлого целиком потерял актуальность.

Например, закончился один документ и начался независимый следующий.

Targeted editing

Нужно, когда требуется изменить одну ассоциацию, сохранив остальные.

Например, факт «Алиса живёт в Риме» должен быть заменён на «Алиса живёт в Париже».

Один глобальный gate не может выборочно удалить Рим, не затронув другую старую информацию.

Одна delta correction не может быстро удалить всё содержимое памяти, которое не перекрывается с текущим ключом.

Отсюда возникает Gated DeltaNet.

18. Gated DeltaNet

Сначала всё старое состояние умножается на глобальный retention gate α_t :

$$S' = \alpha_t S_{t-1}.$$

После этого к уже ослабленному состоянию применяется delta correction:

$$S_t = \alpha_t S_{t-1} + \beta_t (v_t - \alpha_t S_{t-1} k_t) k_t^\top.$$

Раскрывая скобки:

$$S_t = \alpha_t S_{t-1} + \beta_t v_t k_t^\top - \alpha_t \beta_t S_{t-1} k_t k_t^\top.$$

И в operator form:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

Роль α_t

α_t отвечает за **global retention**.

Она действует на всё старое состояние независимо от направления ключа.

Роль β_t

β_t отвечает за **targeted correction** по текущему ключу.

Она одновременно управляет key-directed erase и величиной новой записи.

Разложение по направлениям

Для единичного k_t старое состояние можно мысленно разделить на:

- компоненту, читаемую через k_t ;
- компоненту, ортогональную k_t .

Ортогональная часть после update только умножается на α_t .

По текущему ключу:

$$S_t k_t = (1 - \beta_t) \alpha_t S_{t-1} k_t + \beta_t v_t.$$

Поэтому при $\beta_t = 1$

$$S_t k_t = v_t$$

независимо от α_t .

Это и есть независимость двух механизмов: можно одновременно сильно забыть всё прошлое и точно записать новое значение.

19. Крайние режимы Gated DeltaNet

$$\alpha_t = 1$$

Получается обычная delta rule:

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top.$$

$$\beta_t = 0$$

Новая информация не записывается и key-directed correction отсутствует:

$$S_t = \alpha_t S_{t-1}.$$

Остаётся только глобальный decay.

$$\alpha_t = 0$$

Всё старое состояние удаляется:

$$S_t = \beta_t v_t k_t^\top.$$

При $\beta_t = 1$ после шага остаётся только новая rank-one запись.

Такой режим естественно реализует сброс памяти на границе независимых документов.

$$\alpha_t = 1, \beta_t = 0$$

$$S_t = S_{t-1}.$$

Память полностью заморожена: она не забывает и не записывает.

Важное ограничение

Обычная additive write не получается из этой формулы простым выбором гейтов.

В Gated DeltaNet одна и та же β_t одновременно определяет силу записи и силу key-directed erase. Если выключить erase через $\beta_t = 0$, одновременно выключается и запись.

Поэтому Gated DeltaNet содержит delta rule и режим чистого decay без записи как частные случаи, но не является просто универсальной формулой, включающей вообще все четыре рассматриваемых механизма.

20. Четыре режима управления конечным state

Механизм	Что происходит со старым состоянием	Что записывается	Основная возможность
Additive write	сохраняется полностью	$v_t k_t^T$	только накопление
Fixed decay	всё состояние умножается на γ	$v_t k_t^T$	ограничение lifetime
Delta rule	erase по текущему key	prediction error	targeted correction
Gated DeltaNet	global α_t + erase по текущему key	β_t -correction	независимое управление forgetting и editing

Обычная additive memory похожа на копилку: новые записи только добавляются.

Decay добавляет утечку: старое постепенно исчезает.

Delta rule превращает память в редактируемую ассоциативную структуру: перед записью определяется, что уже находится по этому ключу.

Gated DeltaNet добавляет отдельную ручку общего забывания.

21. Ловушка половинной точности

Для длинного lifetime параметр decay должен быть очень близок к единице:

$$\tau = \frac{1}{1 - \gamma}.$$

Чем больше желаемый τ , тем меньше величина $1 - \gamma$.

Это создаёт опасность при использовании fp16 и bf16: сетка представимых чисел около единицы дискретна.

Если нужное $\gamma < 1$ после округления становится ровно 1, реальное обновление превращается из

$$S_t = \gamma S_{t-1} + \dots$$

в

$$S_t = S_{t-1} + \dots$$

и decay полностью исчезает.

Это особенно неприятная численная ошибка:

- нет NaN;
- нет исключения;
- вычисления продолжаются;
- модель фактически реализует другой transition.

Почему bf16 не обязательно помогает

У bf16 широкий exponent range, но малая точность mantissa.

Для decay около единицы важен не максимальный диапазон значений, а способность различать очень близкие числа вроде 1 и $1 - \epsilon$.

Поэтому «формат имеет большой диапазон» и «формат хорошо представляет маленькое отличие от единицы» — разные свойства.

Чувствительность memory scale

Функция

$$\tau(\gamma) = \frac{1}{1 - \gamma}$$

становится чрезвычайно чувствительной к ошибке γ около единицы.

Небольшая ошибка округления способна заметно изменить lifetime, а округление до $\gamma = 1$ превращает формальный масштаб памяти в бесконечный.

22. Численно устойчивое хранение decay

Один из способов — хранить не γ , а его логарифм:

$$\lambda = \log \gamma.$$

Тогда накопленный decay вычисляется как

$$\gamma^n = \exp(n\lambda).$$

Для переменных гейтов:

$$\prod_j \gamma_j = \exp \left(\sum_j \log \gamma_j \right).$$

Критически важно вычислить **log γ до** того, как γ успел округлиться в низкой точности до единицы. Если сначала сохранить $\gamma = 1$, последующий логарифм уже не восстановит потерянную информацию.

Другой вариант — хранить чувствительные гейты или их накопления в более высокой точности.

Точность самих сумм и экспонент также требует отдельной проверки: переход в log-domain решает проблему представления маленького decay, но не автоматически все проблемы накопленной численной ошибки.

Практические инструменты диагностики

В PyTorch полезно явно проверять gate после преобразования в `float16`, `bfloat16` и `float32`.

`torch.nextafter` позволяет посмотреть ближайшее представимое число к `1` и понять реальное разрешение выбранного dtype около критической области.

Также полезно считать эффективный scale уже из **сохранённого** значения:

$$\tau_{\text{stored}} = \frac{1}{1 - \gamma_{\text{stored}}}.$$

Поддержка dtype библиотекой или GPU не гарантирует высокой производительности. Нужно отдельно различать:

- возможность выполнить операцию;
- наличие аппаратного ускорения данного формата;
- фактическую скорость на конкретном GPU.

На старых поколениях GPU bf16 может быть функционально доступен, но не иметь такого же быстрого аппаратного пути, как fp16.

23. Проверки корректности реализации

Несколько предельных случаев позволяют быстро обнаруживать ошибки в формулах и коде.

Проверка decay

При

$$\gamma = 1$$

decayed operator должен превратиться в обычный additive operator.

Проверка delta rule

При

$$\beta = 0$$

состояние не должно изменяться из-за записи.

Для единичного ключа при

$$\beta = 1$$

после correction обязательно должно выполняться

$$Sk = v$$

для только что записанного ключа.

Ортогональные ключи

Если

$$k_i^\top k_j = 0,$$

correction по k_i не должно менять чтение по k_j .

Проверка Gated DeltaNet

При $\alpha = 1$ должна получаться delta rule.

При $\beta = 0$ должен оставаться только global decay.

При $\alpha = 0$ не должно сохраняться ничего из старого состояния.

Такие предельные случаи полезнее проверки нескольких случайных чисел, потому что следуют непосредственно из математической структуры оператора.

24. State equation, operator, evaluation algorithm и architecture — разные уровни

Похожие recurrence ещё не означают одинаковую архитектуру.

Полезно разделять четыре уровня.

State equation

Правило перехода

$$S_{t-1} \rightarrow S_t.$$

Например, decay, delta rule или gated delta update.

Operator

Полное отображение последовательности входов в последовательность выходов, включая запись и чтение.

Одинаково выглядящие state equations могут входить в разные operators.

Evaluation algorithm

Способ вычислить тот же operator:

- recurrent;
- parallel;
- chunkwise;
- scan;
- FFT;
- другие факторизации.

Два алгоритма могут выдавать точно одинаковые значения, имея совершенно разную стоимость на железе.

Full architecture

Вся модель вокруг оператора:

- входные и выходные projections;
- convolution;
- normalization;
- gates;
- feature maps;
- residual paths;
- порядок блоков;
- число слоёв;
- размеры состояний.

Совпадение одного внутреннего механизма не делает две полные архитектуры одинаковыми.

25. Три степени сходства моделей

Exact identity

Две формы на одинаковом входе дают буквально одинаковые численные результаты.

Пример — точные recurrent, parallel и chunkwise формы одного decayed operator.

Разные algorithms одного operator

Математическое отображение одно, но способы вычисления различаются по параллелизму, памяти и пригодности для training/inference.

Architectural similarity

Архитектуры используют сходную идею, например input-dependent forgetting.

Это полезное наблюдение, но оно не доказывает математической эквивалентности.

Поэтому утверждение вида «модель эквивалентна Transformer» имеет смысл только после уточнения: **на каком уровне эквивалентна?**

26. Как рассматриваемые идеи проявляются в реальных архитектурах

RetNet

Чистый пример decayed retention и нескольких точных форм вычисления одного оператора:

- parallel;
- recurrent;
- chunkwise.

Особенно полезен для понимания различия между математической операцией и алгоритмом её вычисления.

Mamba

Относится к selective state space models.

Параметры динамической системы зависят от входа, поэтому модель может адаптивно решать, какую информацию пропускать через состояние.

Input-dependent decay — только одна родственная идея; наличие такого гейта не делает произвольную модель Mamba.

Mamba-2 и SSD

Structured State Space Duality устанавливает более сильную формальную связь между определённым классом state-space operators и структурированными матрицами, родственными attention.

Это утверждение об определённых классах операций и их представлении, а не утверждение «Mamba и Transformer — одна и та же полная архитектура».

DeltaNet

Использует delta rule как механизм редактирования fast state.

Ключевая инженерная составляющая — возможность точного эффективного chunkwise-вычисления, несмотря на зависимость corrections от порядка.

Gated DeltaNet

Добавляет к delta rule независимый global retention gate α_t .

Получаются две разные степени свободы:

- глобально забывать;
- локально редактировать.

RWKV-7

Использует более обобщённый delta-like механизм с гейтами по отдельным координатам.

При обсуждении RWKV важно указывать версию: семейство существенно менялось, поэтому утверждение про одну версию нельзя автоматически переносить на все остальные.

Hyena

Представляет другую ветвь — длинные convolution operators.

Длинную свёртку можно эффективно вычислять при обучении, например алгоритмами на основе FFT. Но из этого ещё не следует дешёвая авторегрессивная генерация.

Для генерации по одному токenu важен другой вопрос: существует ли эквивалентное конечное recurrent state.

Если существует компактная recurrence, история может быть сведена к фиксированному состоянию.

Если нет, очередной token может требовать работы с длинной историей.

Таким образом, стоимость training и стоимость autoregressive inference нельзя выводить друг из друга только по названию оператора.

27. Основные инженерные компромиссы конечной памяти

Fixed state против KV-cache

Fixed state:

- не растёт с длиной последовательности;
- вынужден сжимать историю;
- имеет ограниченную ёмкость;
- подвержен интерференции.

KV-cache:

- растёт с длиной;
- хранит прошлые K и V отдельно;
- не создаёт именно того bottleneck, который возникает из-за сжатия всей истории в одну матрицу.

Decay

Решает вопрос lifetime.

Большое retention сохраняет полезную информацию, но долго сохраняет и накопленные помехи.

Сильное forgetting очищает память, но одновременно уничтожает потенциально нужные старые факты.

Delta rule

Решает вопрос targeted correction.

Она позволяет заменять ответ по текущему ключу, но коррекция затрагивает похожие ключи.

Gated DeltaNet

Разделяет global forgetting и targeted editing, но конечное состояние всё равно остаётся конечным: две ручки улучшают управление ёмкостью, а не делают ёмкость бесконечной.

Параллелизация

Простота recurrent update не гарантирует дешёвого parallel training.

Scalar decay легко переставляется и факторизуется.

Key-dependent matrix erase не коммутирует, поэтому требует более сложных алгоритмов вроде chunkwise WY-представления.

Численная точность

Даже математически правильный gate может исчезнуть после округления.

Для долгих time scales формат хранения параметров становится частью определения реально выполняемой модели.

28. Примеры применения механизмов

Повтор одной и той же информации

При additive write повторный факт продолжает накапливаться в S .

При delta rule, если память уже выдаёт правильный ответ, prediction error равна нулю и дополнительная запись не требуется.

Изменение факта

«Алиса живёт в Риме» → «Алиса живёт в Париже».

Additive write склонна создавать смесь старого и нового.

Decay постепенно ослабит Рим, но вместе с другой старой информацией.

Delta rule непосредственно исправит чтение по соответствующему ключу.

Похожие, но разные факты

Если два различных факта получают близкие ключи, correction одного изменит чтение второго.

Это случай, где требуется хорошая геометрия learned keys, достаточная размерность и корректное управление памятью.

Граница документов

Если всё прошлое стало нерелевантно, targeted correction недостаточно: старые компоненты, ортогональные новым ключам, могут жить дальше.

Input-dependent global gate способен почти обнулить их за один шаг.

Gated DeltaNet может тем же шагом сбросить старое через α_t и записать новый факт через β_t .

Длинный контекст

Успешный needle test показывает способность достать конкретную далёкую информацию.

Для проверки способности одновременно хранить большое число ассоциаций нужен многозапросный тест вроде MQAR.

Что читать дальше

Bernard Widrow, Marcian E. Hoff — *Adaptive Switching Circuits*, 1960. Ради исторического происхождения LMS/delta rule и понимания online-коррекции линейной модели по ошибке одного примера. Работа доarXiv-эпохи, сетевой ссылки нет.

[arXiv:2106.06295](#) - Kazuki Irie, Imanol Schlag, Róbert Csordás, Jürgen Schmidhuber — *Going Beyond Linear Transformers with Recurrent Fast Weight Programmers*, 2021. Ради интерпретации состояния как fast weights и перехода от простого накопления к более выразительным правилам быстрого обновления памяти.

[arXiv:2102.11174](#) - Imanol Schlag, Kazuki Irie, Jürgen Schmidhuber — *Linear Transformers Are Secretly Fast Weight Programmers*, 2021. Ради связи linear attention с программируемой fast-weight memory и идеи читать состояние перед его коррекцией.

[arXiv:2312.04927](#) - Simran Arora et al. — *Zoology: Measuring and Improving Recall in Efficient Language Models*, 2023. Ради MQAR и понимания того, почему однозапросные тесты длинного контекста могут

скрывать ограниченную recall-capacity эффективных архитектур.

[arXiv:2307.08621](#) - Yutao Sun et al. — *Retentive Network: A Successor to Transformer for Large Language Models*, 2023. Ради retention/decay и точной связи parallel, recurrent и chunkwise форм одного оператора.

[arXiv:2312.00752](#) - Albert Gu, Tri Dao — *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, 2023. Ради selective state space models и понимания того, как вход может управлять динамикой конечного состояния.

[arXiv:2406.06484](#) - Songlin Yang et al. — *Parallelizing Linear Transformers with the Delta Rule over Sequence Length*, 2024. Ради DeltaNet, последовательного delta update и алгоритмов точного chunkwise-вычисления с компактным представлением цепочки key-dependent transformations.

[arXiv:2405.21060](#) - Tri Dao, Albert Gu — *Transformers are SSMS: Generalized Models and Efficient Algorithms Through Structured State Space Duality*, 2024. Ради Structured State Space Duality, Mamba-2/SSD и аккуратного понимания того, на каком именно уровне state-space operators могут быть связаны с attention-like матрицами.

[arXiv:2412.06464](#) - Songlin Yang et al. — *Gated Delta Networks: Improving Mamba2 with Delta Rule*, 2024. Ради разделения global forgetting и targeted correction через независимые α - и β -гейты.

[arXiv:2302.10866](#) - Michael Poli et al. — *Hyena Hierarchy: Towards Larger Convolutional Language Models*, 2023. Ради альтернативной линии длинных convolution operators и различия между быстрым параллельным training и стоимостью авторегрессивной генерации.